# Data Science & AI

Facultat de Matemàtiques i Informàtica

## Session 02 · Working environment and programming with AI

Xavier Baró

xbaro@ub.edu

xbaro@ub.edu
Departament de Matemàtiques i Informàtica de la Universitat de Barcelona



I am an Associate Professor at the Mathematics & Computer Science Department, **Universitat de Barcelona**, where I work on **machine learning** and statistical pattern recognition, with applications to looking at people.

# Today


1
Check
setup works
2
uv
one project
3
Notebooks
kernel & state
4
Git & GitHub
save & share
5
AI assistants
Copilot & Claude
6
Lab
put it together

# 1 · Check your setup

# Green or red?

## 1. Run the check notebook

- Open `verify_setup.ipynb` in VS Code
- Kernel: `.venv` · **Run All**

## 2. All ✅ and you see the plot?

- **Green post-it** on your laptop
- Continue with the class

## 3. Something red?

- **Red post-it**
- We come to you: show us the message

## 4. Most common blockers

- Wrong kernel (not `.venv`)
- Terminal opened before installing uv
- Git not installed or not signed in

# 2 · uv: one project, one environment

# A project is a folder

| | | |
|---|---|---|
| 📁 `postgrau-env/` | | |
| 📁 `.venv/` | the private environment | never edit, never share |
| 📄 `.python-version` | which Python (3.12) | commit |
| 📄 `pyproject.toml` | what the project needs | commit |
| 📄 `uv.lock` | the exact versions installed | commit |
| 📄 `verify_setup.ipynb` | your notebook | commit |

# Add a library, live

Let's add a plotting library and look at what changes:

```
uv add seaborn              # I need this library
uv remove seaborn           # I no longer need it
uv sync                     # rebuild the environment from uv.lock
uv add --dev pytest         # dev tool, used in the lab
```

# pyproject.toml vs uv.lock

## pyproject.toml

- What **you** ask for
- Human-readable, short
- `pandas`, `matplotlib`...

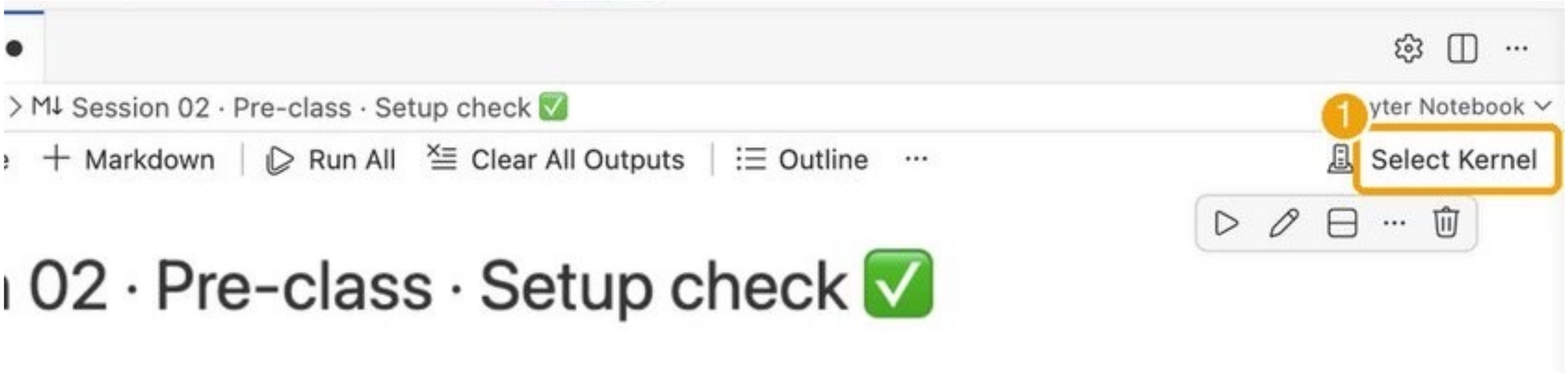
## uv.lock

- What **was installed**, exact versions
- Written by `uv`, never by hand
- Same lock = same environment on any laptop

**Commit both. Never commit `.venv`.**

# 3 · Notebooks in VS Code

# Choosing the kernel (1/2)



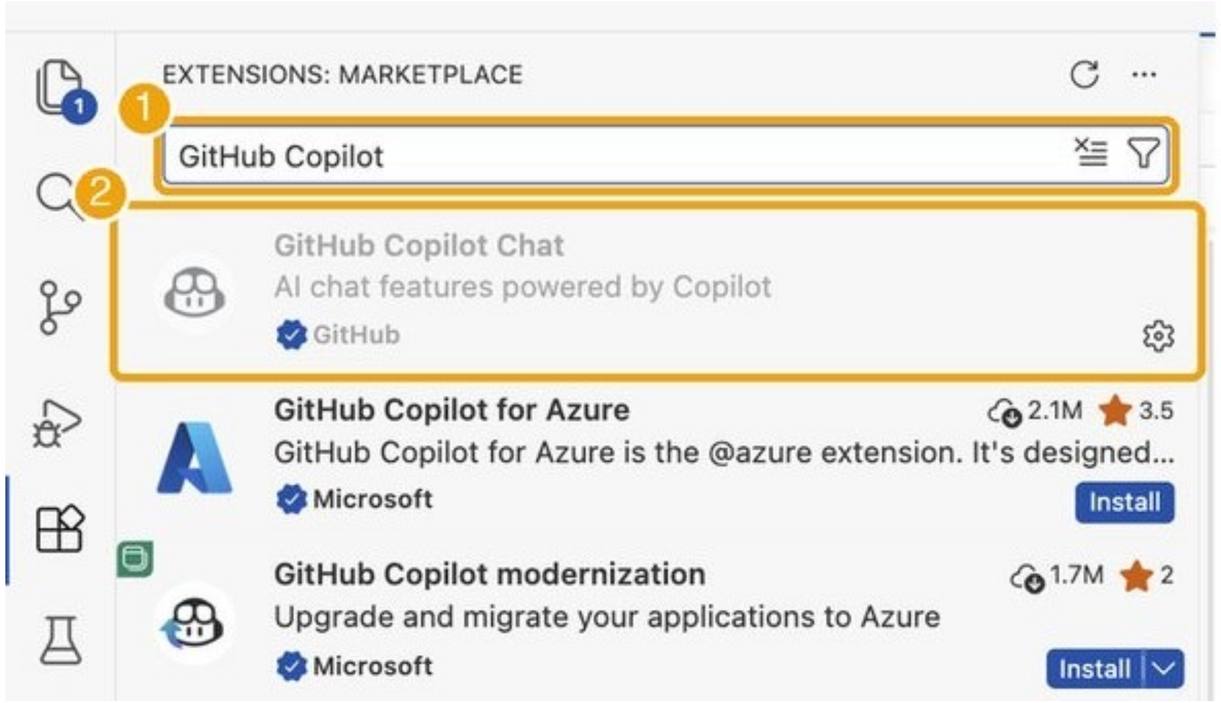

Open the notebook and click Select Kernel (top right).

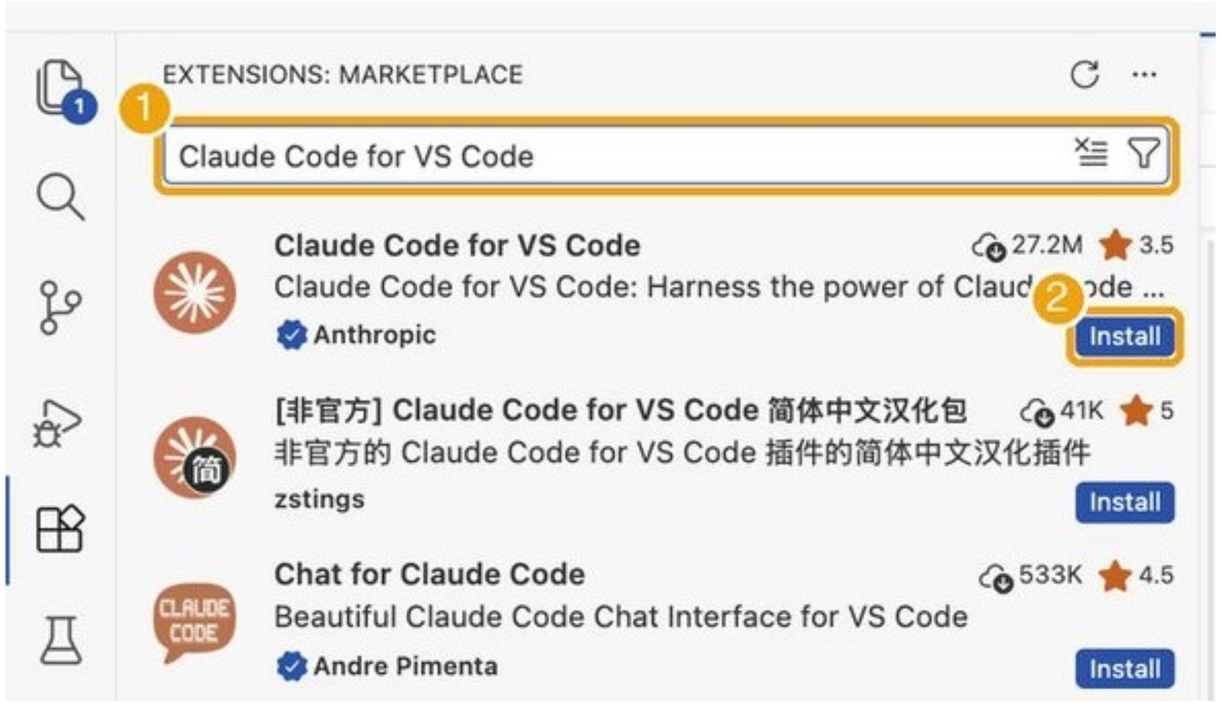
# Choosing the kernel (2/2)




Python Environments... then pick the entry that says .venv (Recommended).

# Install the extensions



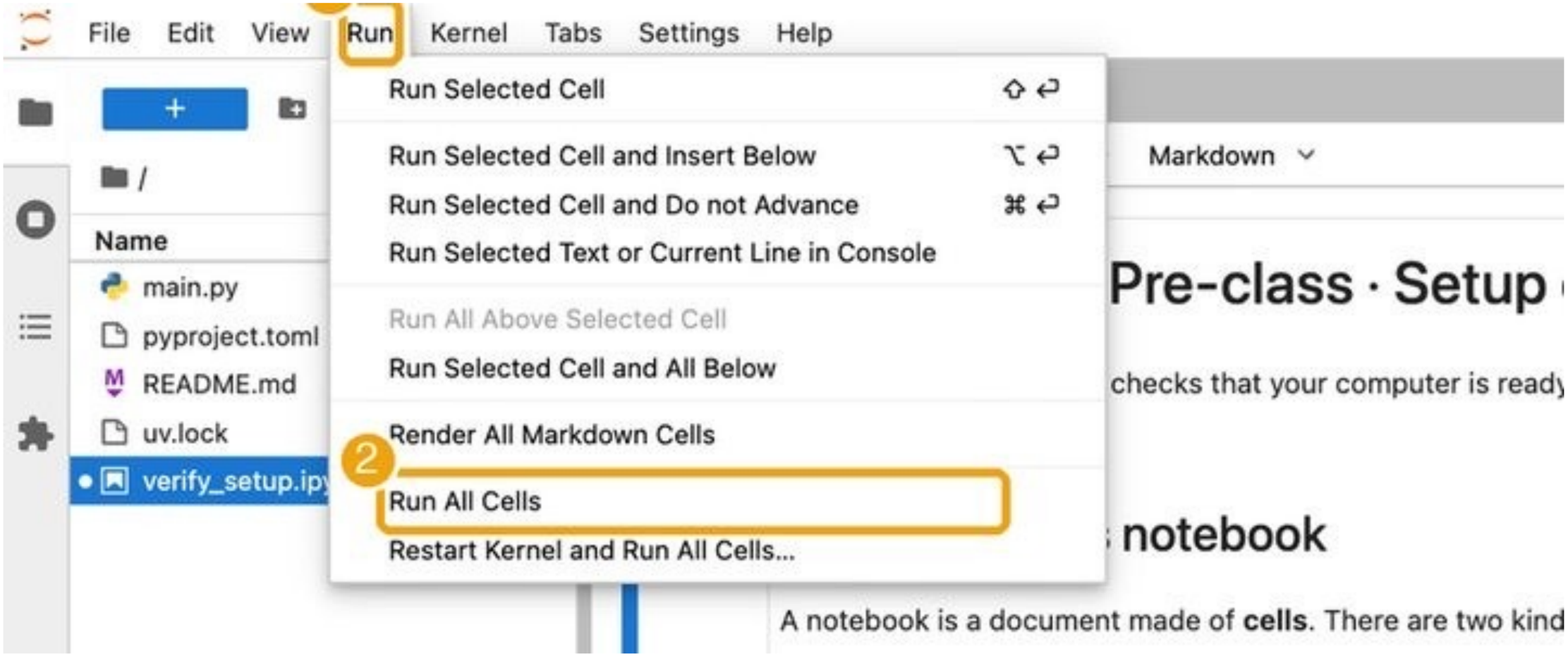

Extensions icon, search by name, Install. Claude Code: install only; credentials come later.

# Kernel problems? Use JupyterLab

Plan B, in the browser, same project environment:

```
uv add --dev jupyterlab
uv run jupyter lab
# a browser tab opens: open verify_setup.ipynb and Run All
```

# JupyterLab: Run All Cells




Run, then Run All Cells. Same checks, same environment.

# The hidden-state trap

```
[1]  price = 10
```

```
[3]  total = price * 3
     total → 60
```

```
[2]  price = 20
```

You ran the cells in the order **1** → **2** → **3** (see the numbers),
but on screen they read top to bottom.

**Kernel memory**

```
price = 20
total = 60
```

what the kernel remembers,
not what you see

**60 ≠ 10 × 3 ? Hidden state!**

**Cure: Restart + Run All**

# The rule

**Before you share a notebook: Restart and Run All. If it fails, it is broken.**

# Notebook or .py file?

## Notebook

- Explore a dataset
- Try ideas, make plots
- Explain results to others

## `.py` file

- Code you reuse (a cleaning function)
- Code you **test**
- Easy to review in Git

**Today's lab uses both: the function in `clean.py`, the story in the notebook.**

# 4 · Git and GitHub

# Local and remote

## 💻 Your laptop

local repository




C exists **only here**

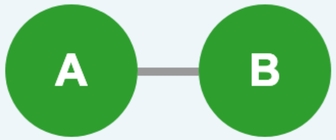
**push →**

**← pull**

clone: first copy from GitHub to laptop

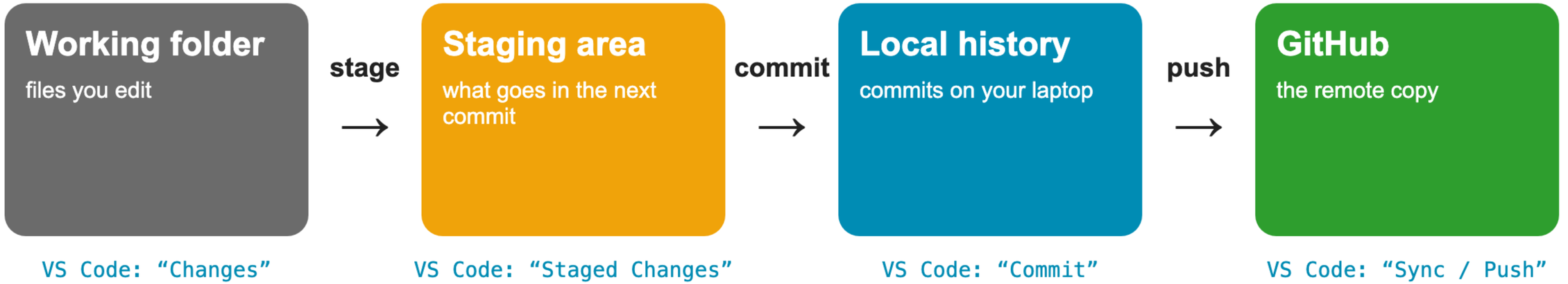
## ☁️ GitHub

remote repository




**No push, no safe.** **Commits that are not pushed live only on your laptop.**

# From edit to GitHub




**Commit** = a save point with a message (“Add cleaning function”)

**Push** = send your save points to GitHub (the backup + how others see them)

# Three ways to use GitHub



## Browser
github.com

- Upload files, edit a file
- Like a “Dropbox”
- No install needed
- Best for: quick fixes



## GitHub Desktop
visual app

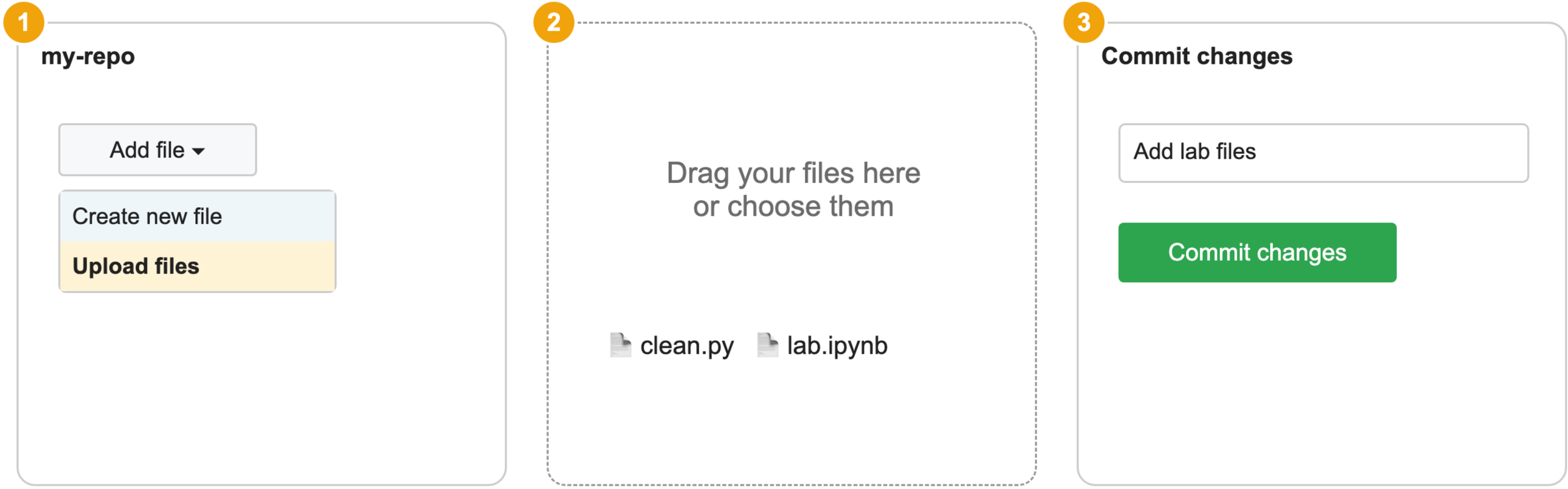
- Changes list, one button
- Clone, commit, push
- Windows and macOS
- Best for: starting out



## VS Code
inside your editor

- Source Control panel
- Same editor as your code
- Windows, macOS, Linux
- Best for: daily work

# In the browser: like a Dropbox



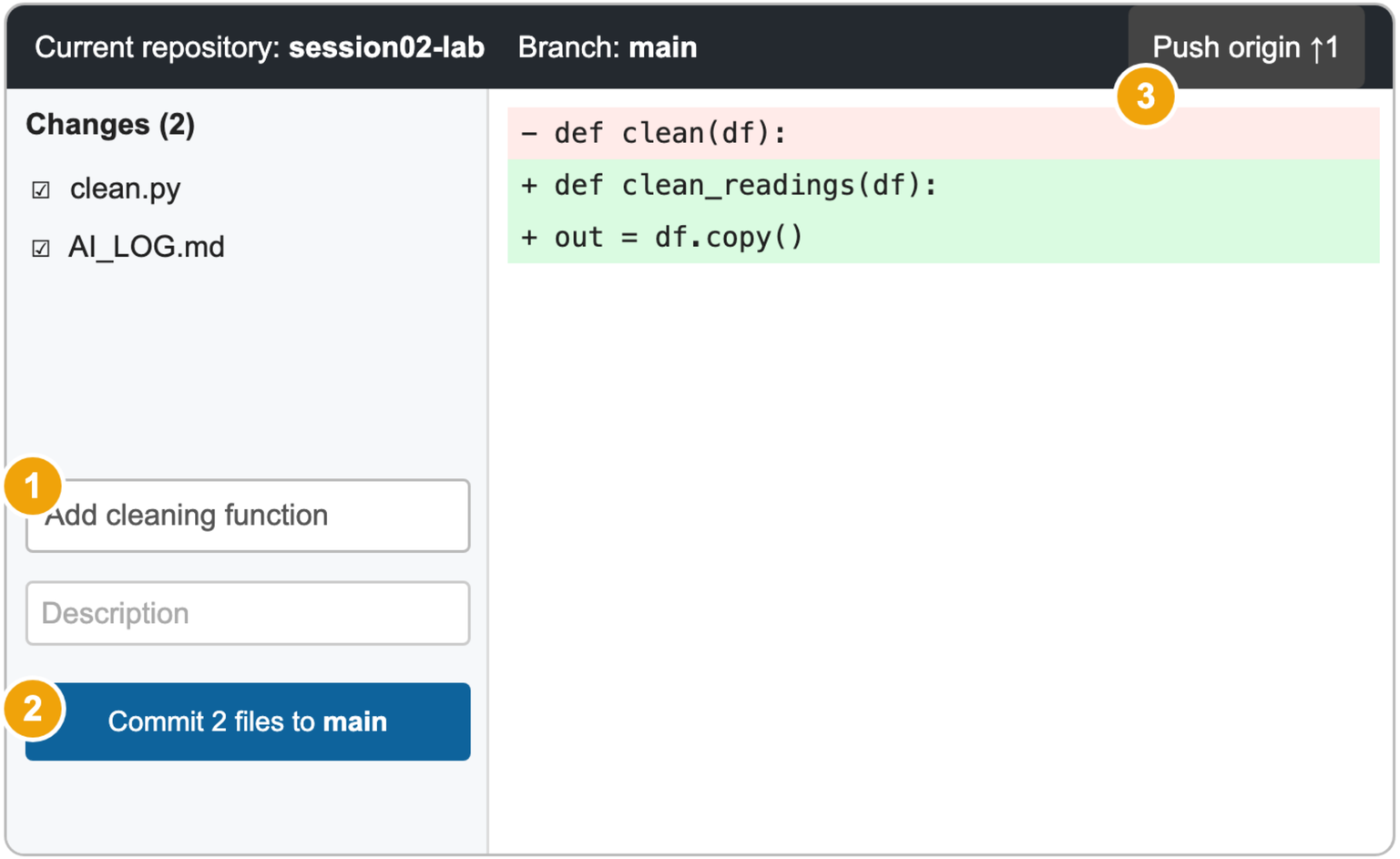

**Add file → Upload files**

**Drag the files** into the page

**Write a message → Commit**

In the browser, “commit” also saves to GitHub straight away: no push needed.

# GitHub Desktop



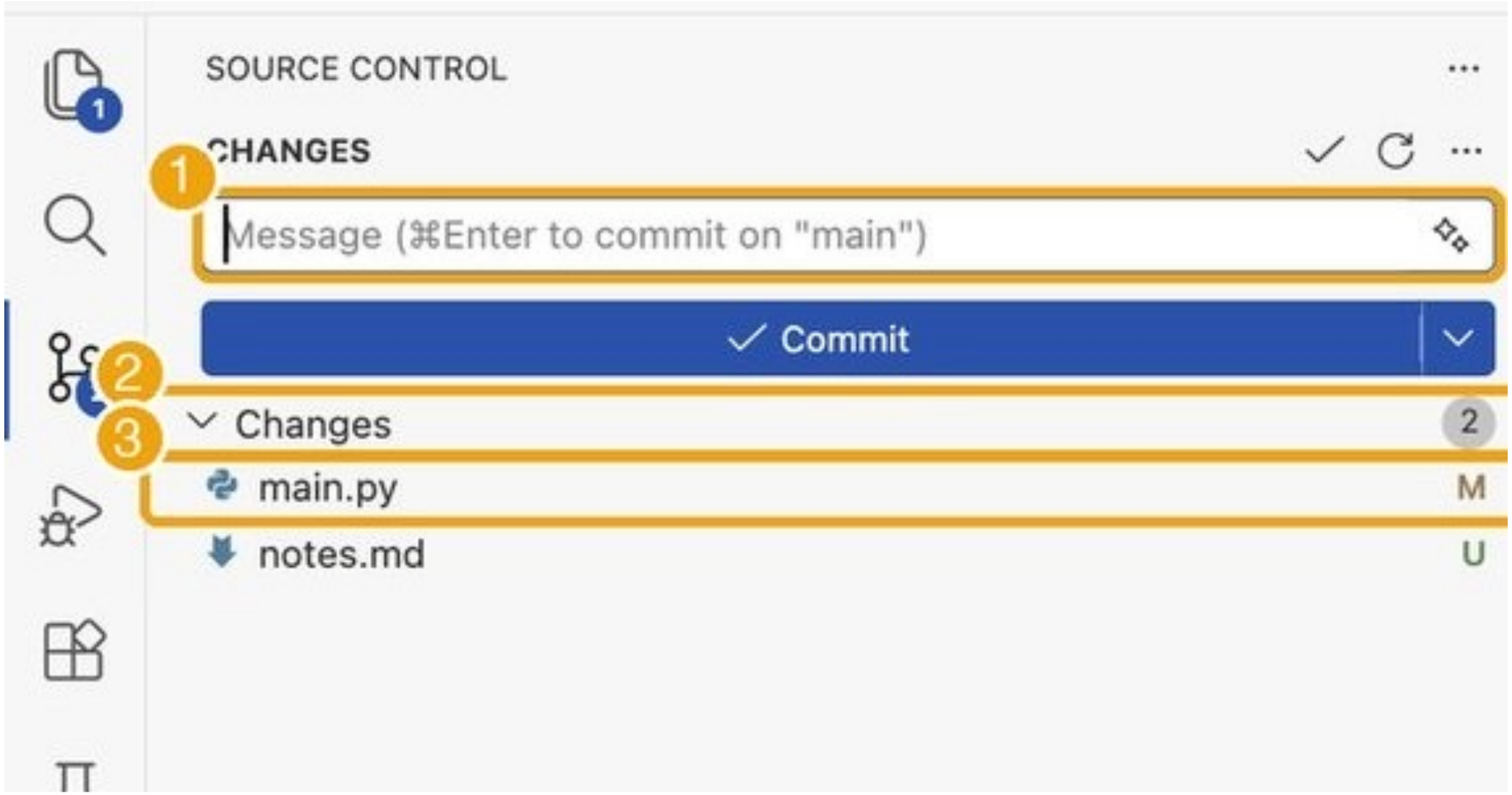

1 Tick the files that go in this commit

2 Write a summary, then **Commit to main** (save point on your laptop)

3 Click **Push origin** to send it to GitHub

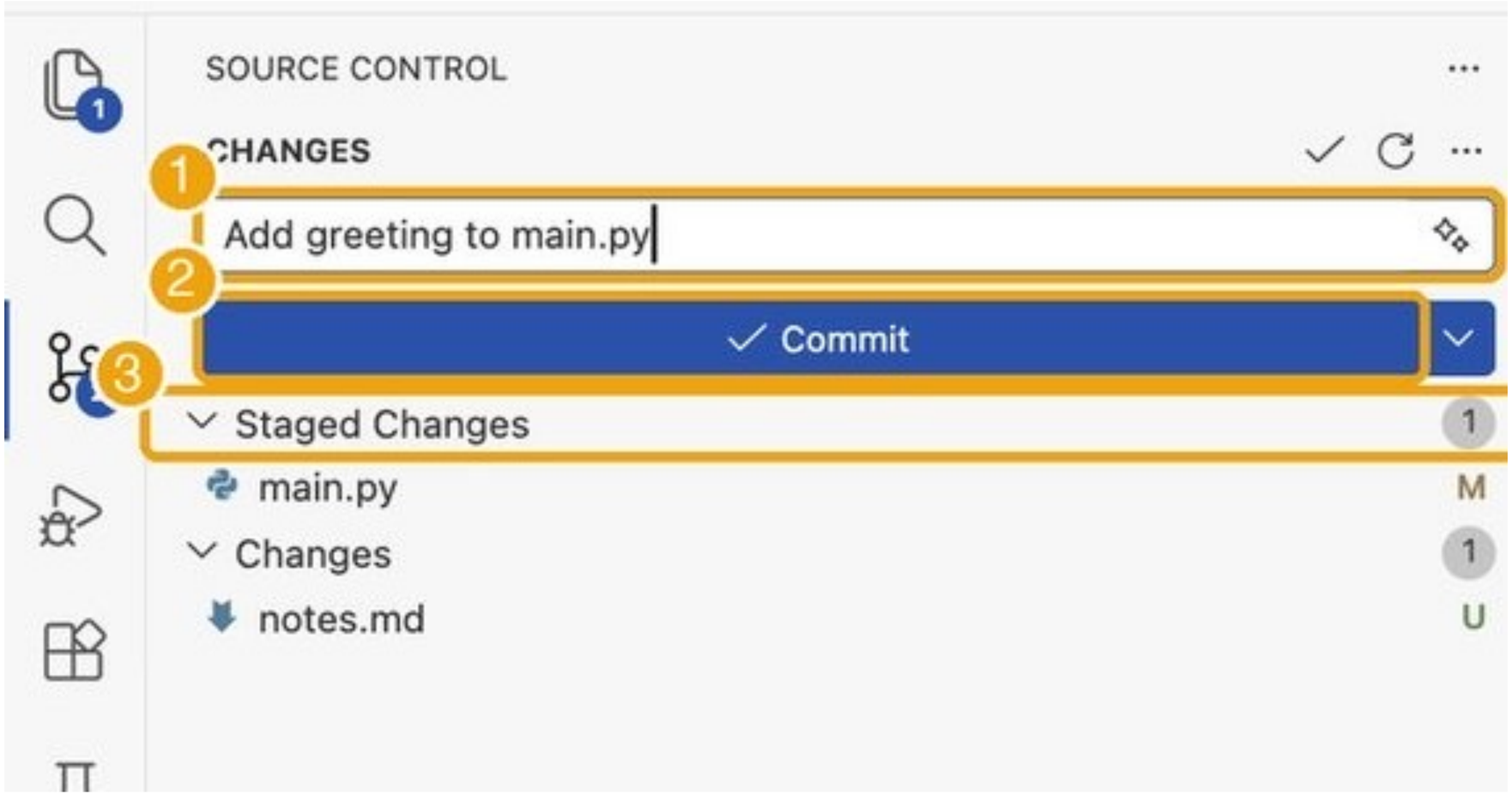
The right-hand side shows the diff: red = removed, green = added.

# VS Code: Source Control



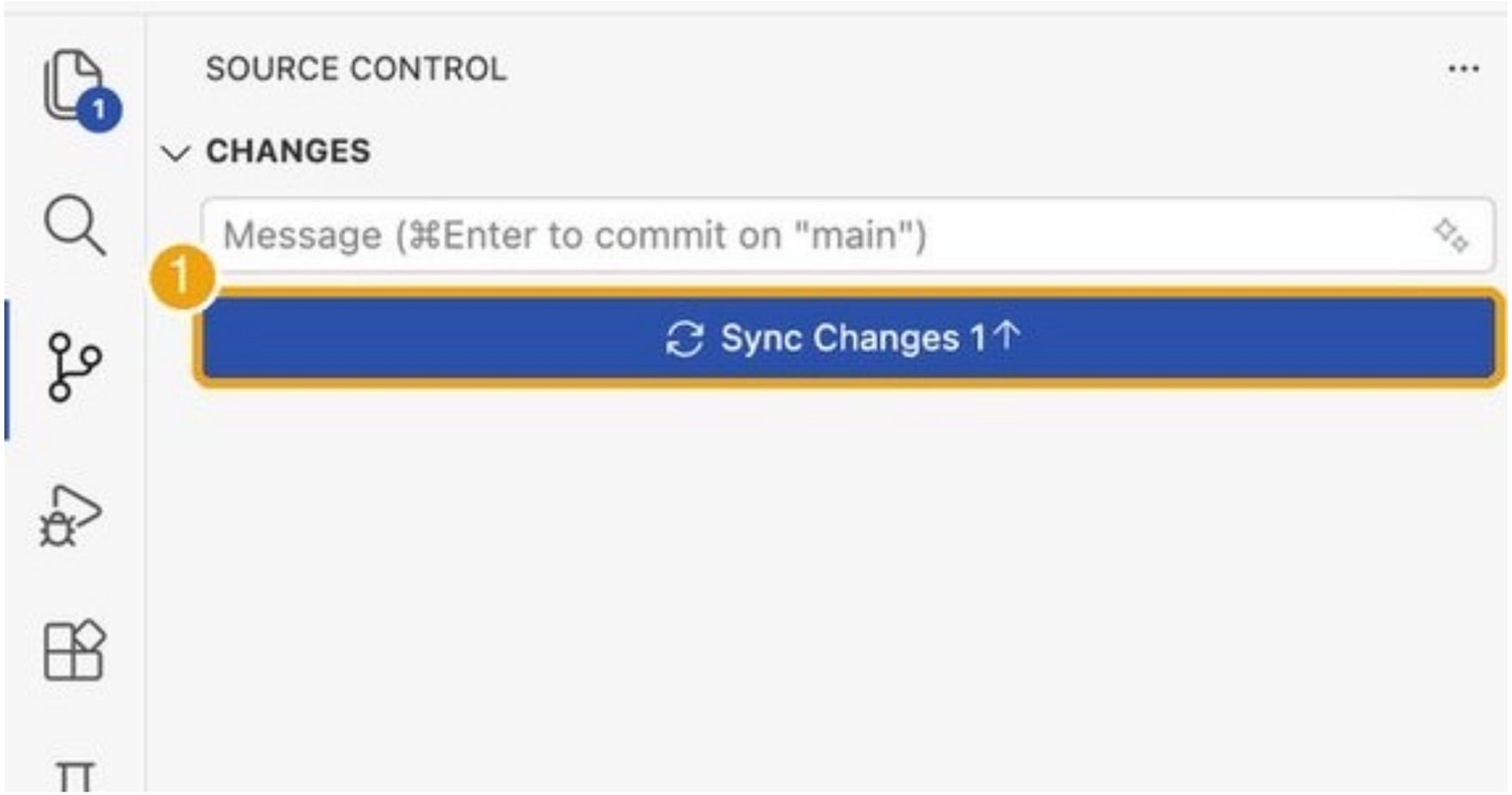

(1) message box, (2) Changes list: M = modified, U = new file.

# VS Code: stage and commit



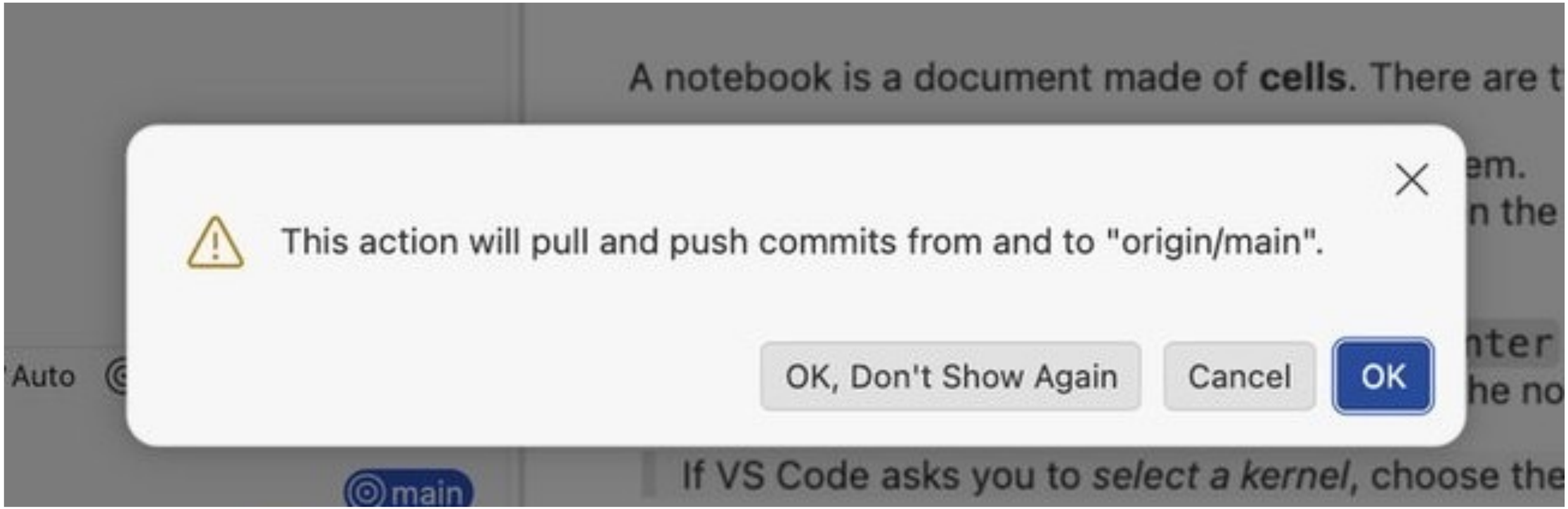

Hover a file and click + to stage it. Write a message (1), then Commit (2).

# VS Code: commit is not push



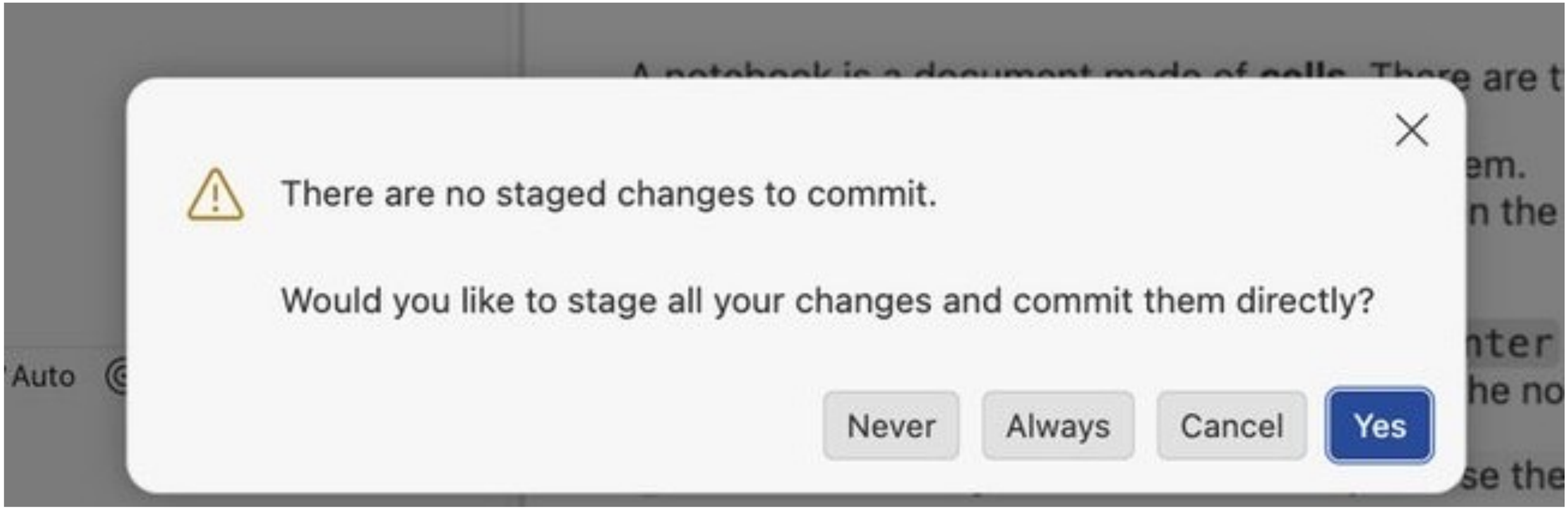

After the commit: Sync Changes 1 up arrow = one commit not on GitHub yet.

# VS Code: confirm the sync



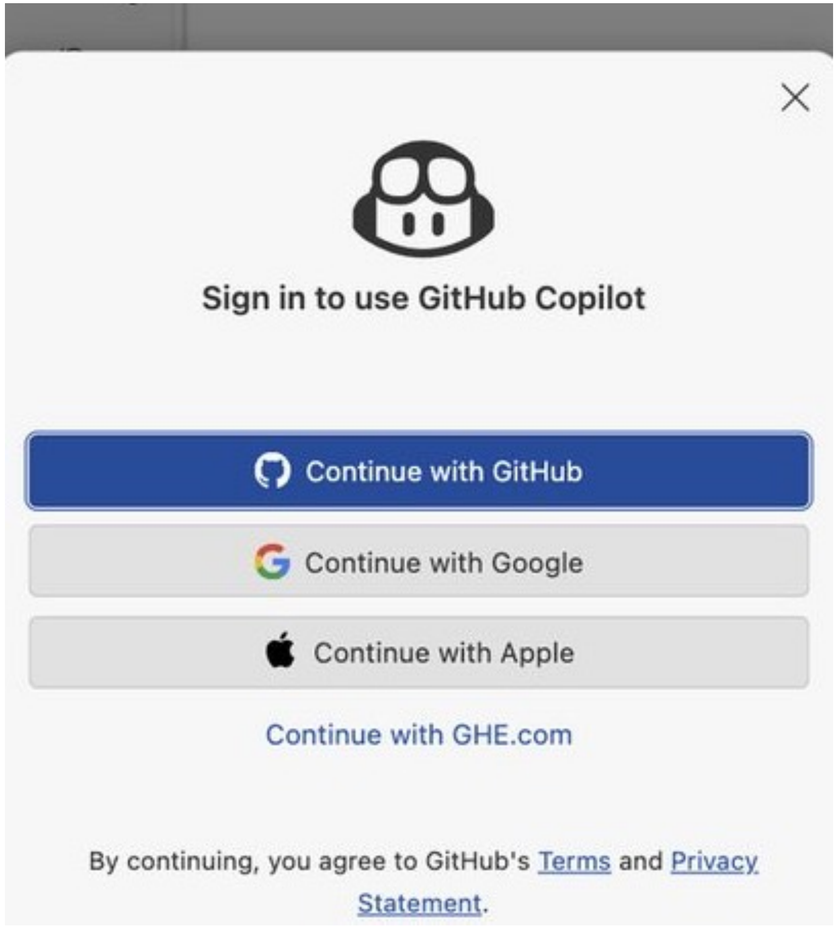

First time: click OK.

# VS Code: a commit with nothing staged



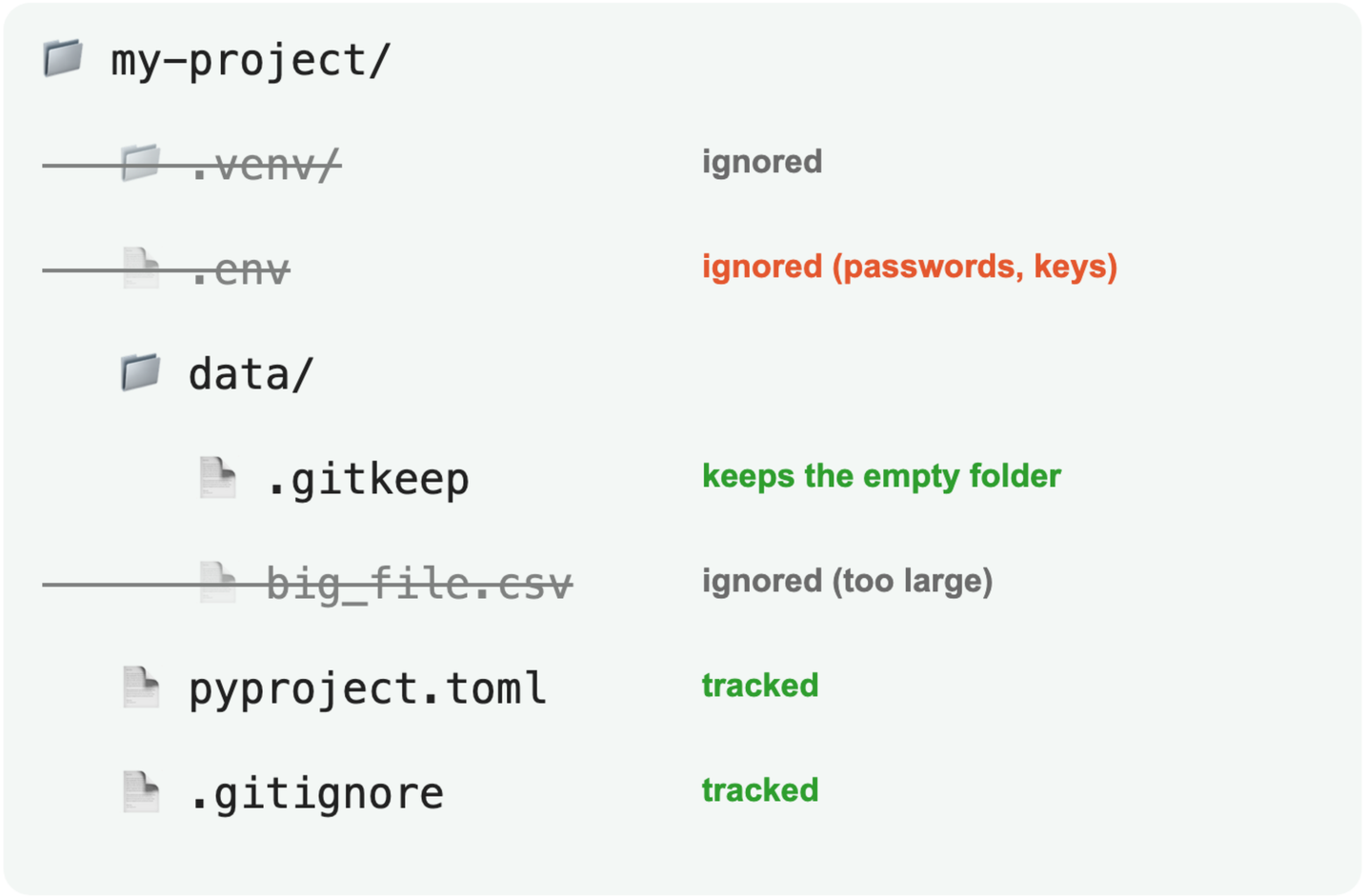

Never / Always / Cancel / Yes. Yes is fine for now.

# Sign in to GitHub from VS Code




Choose Continue with GitHub: your browser opens to authorize. No password in VS Code.

# .gitignore and .gitkeep




**.gitignore**

```
.venv/
.env
data/*
!data/.gitkeep
```

**.gitignore** = “do not track these”.

**.gitkeep** = an empty file so Git keeps an **empty folder** (Git only tracks files).

# First time on a laptop

Tell Git who you are (once per computer):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

# Your first repository (guided)

## 1. Create the repo

- GitHub Desktop: **File** → **New repository…**
- Name: `session02-lab`, keep it **private**

## 2. Make it a uv project

- `uv init --python 3.12` in that folder
- `uv add pandas matplotlib`
- `uv add --dev ipykernel pytest`

## 3. Commit and publish

- Summary: Initial project
- **Commit to main**, then **Publish repository**

## 4. Check it online

- Open the repo on github.com
- You should see `pyproject.toml` and `uv.lock`

# Organizing your folders

📁 **postgrau/**

📁 **posgradoDSAI2026/**

**CLONE** of the course repository

- you do **not** edit it
- before each session:
  **`git pull`** to get new material

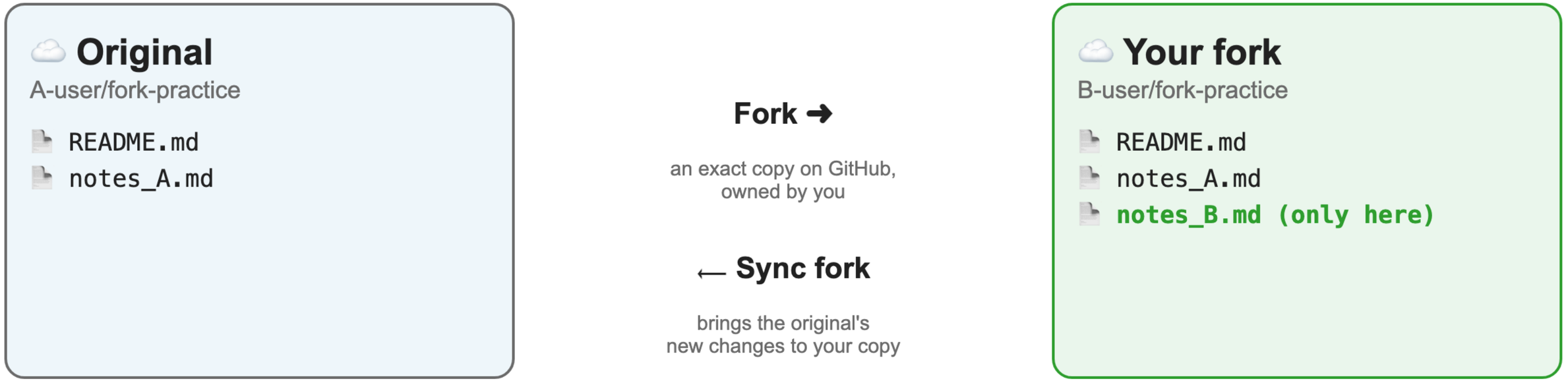
📁 **my-work/**

**YOUR** own repository

- your notebooks, code, exercises
- you commit and **push** here
- copy files from the clone when needed

Two folders = no clashes between your changes and the new material. **The simplest option.**

# Advanced: what is a fork?




You can change the fork freely: **nothing goes back** to the original unless you ask for it.

# Fork exercise (1/3): A creates the original

## 1. Create the repository

- github.com/new: `fork-practice`, **Public**
- Tick Add a README file

## 2. Open the web editor

- On the repository page press the `.` key
- It is VS Code in your browser

## 3. Write a file

- New file `notes_A.md`, write something
- Source Control: message, then commit

## 4. Check it online

- The file is on the repository page
- Public, so a classmate can fork it

# Commit: main or a branch?

- The editor can offer to commit to **main** or to a new **branch**
- We **do not work with branches** in this course: always choose **main**
- In the web editor a commit goes **straight to GitHub**: there is no separate push
- On your laptop (VS Code, GitHub Desktop) you **do** need the push

**No branches: always main. On the laptop: no push, no safe.**

# Fork exercise (2/3): B forks and adds a file

## 1. Fork

- Open A's repository, click **Fork**
- **Create fork**: B now has a copy

## 2. Compare

- Same files as the original
- A fork starts as an exact copy

## 3. Create a second file

- `notes_B.md` in B's fork, commit to **main**
- Check both repositories

## 4. Observe

- `notes_B.md` exists **only in the fork**
- The original has not changed

# Fork exercise (3/3): synchronize

## 1. A changes the original

- A creates `notes_A2.md` in the original
- Commit to **main**

## 2. B's fork is behind

- It does **not** have `notes_A2.md`
- Forks do not update by themselves

## 3. B syncs

- On B's fork: **Sync fork → Update branch**
- Result: `notes_A2.md` **and** `notes_B.md`

## 4. Conflicts?

- Same file changed in both places
- Stop and ask: it needs a decision

# The whole exercise

**A**

Creates fork-practice with notes_A.md

**B**

Forks it: same content as the original

**B**

Creates notes_B.md: only in the fork

**A**

Creates notes_A2.md in the original

**B**

Sync fork → Update branch: has all three

A = owner of the original · B = owner of the fork (work in pairs; you cannot fork your own repository)

# Another option: work in a fork of the course repo

## Two folders (simplest)

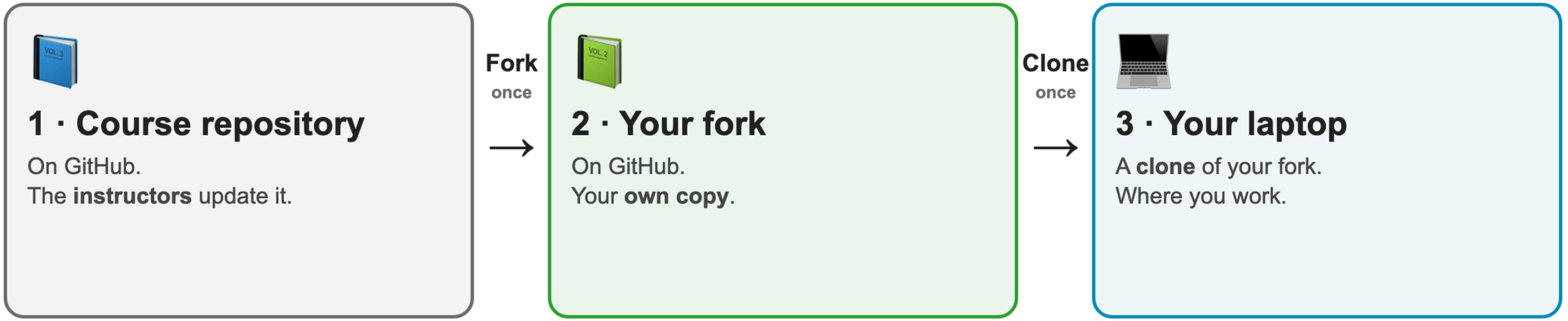
- A clone of the course repo + your own repo
- Before a session: `git pull` in the clone
- Your work and the course material are separated

## Fork of the course repo

- One folder: your fork, cloned
- Before a session: **Sync fork**, then `git pull`
- Your work lives next to the material

**Not sure? Start with the two folders.**

# Fork, without jargon



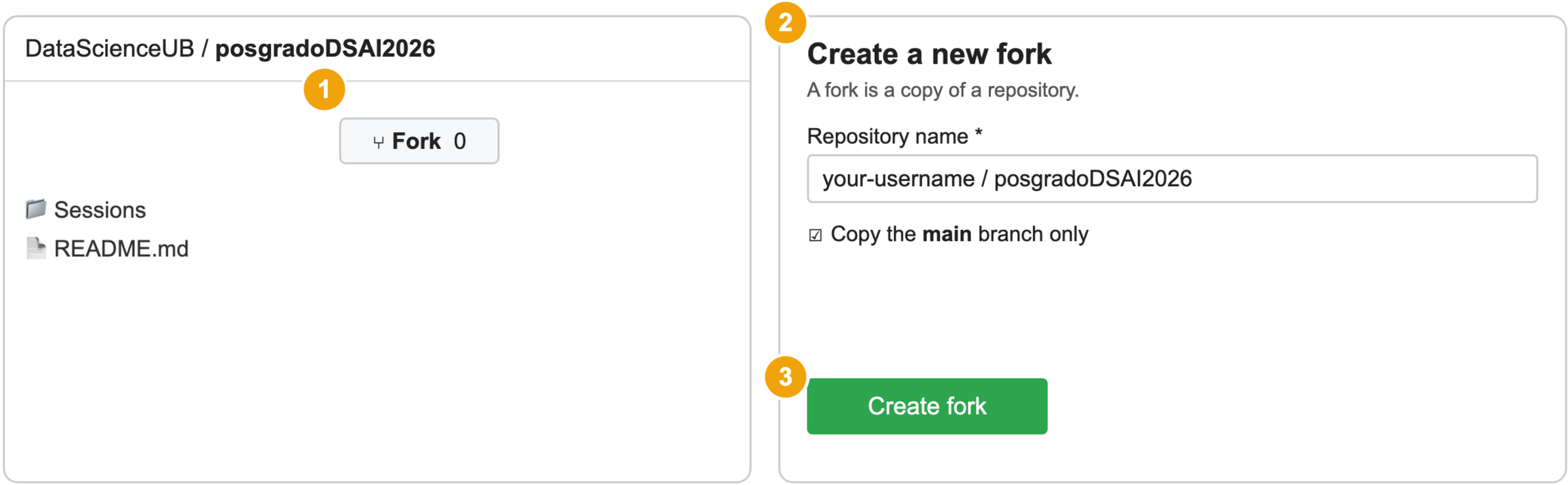

**Every session: updates come DOWN**

1 → 2: **Sync fork** (on github.com)
2 → 3: **Pull** (on your laptop)

**Your own work goes UP**

3 → 2: **Commit** and **Push**
It stays in **your fork**, never in the original

# Fork the course repository




1 On the course repository page, click **Fork** 2 Keep the name 3 Click **Create fork**. You land on **your** copy: `your-username/posgradoDSAI2026`

Illustration: GitHub's pages may look slightly different.

# Get new material: sync fork, then pull

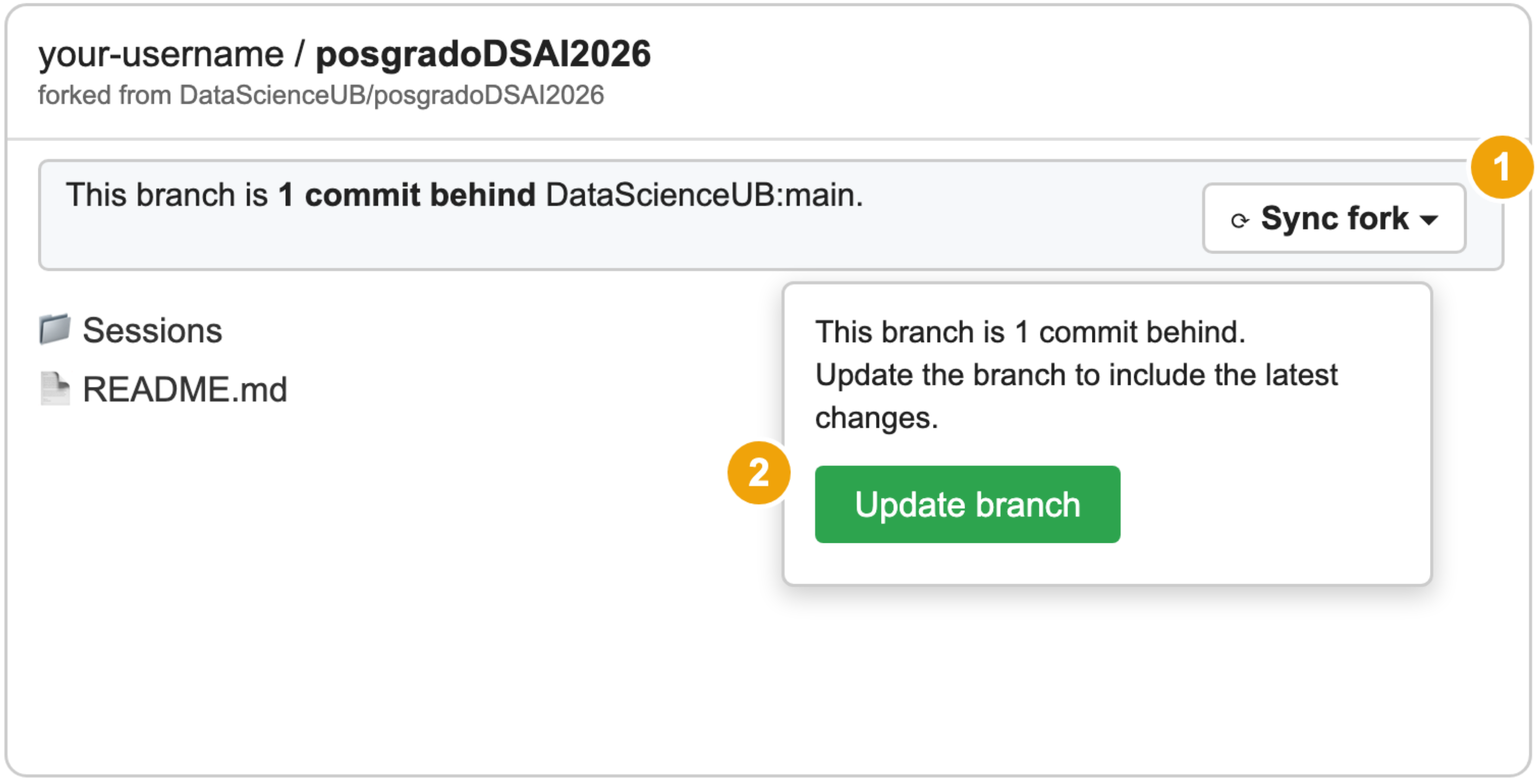



Illustration: GitHub's pages may look slightly different.

1 Open **your fork** on github.com: a banner says it is **behind** the original. Click **Sync fork**.

2 Click **Update branch**. The new files arrive in your fork.

3 Then, on your laptop: **pull** (`git pull`, or **Sync Changes** in VS Code).

If the banner says *"up to date"*, there is nothing new.

# Your turn: fork and sync

## 1. Fork (if you have not)

- Course repository → **Fork** → **Create fork**
- Clone **your fork** to your laptop

## 2. Wait for the test file

- The instructor adds `sync_test.md` to the **course repository**
- Your fork does not have it yet

## 3. Sync your fork

- github.com → your fork → **Sync fork** → **Update branch**
- The banner says 1 commit behind

## 4. Pull and check

- On your laptop: **pull** (Sync Changes / Pull origin)
- Open `sync_test.md` in your folder ✅

# Notebooks and Git: the problem

- A notebook is a **JSON file**: code + outputs + metadata
- Re-running changes **outputs** and `execution_count`, even if the code is the same
- Plots are embedded as **images** inside the file: huge, unreadable diffs
- Two people editing one notebook → painful **merge conflicts**

**A one-line change can look like a 58-line change.**

# Notebooks and Git: what to do

## Good habits

- Clear outputs before committing (Clear All Outputs)
- Move reusable code to `.py` files
- Small notebooks, one question each
- Restart and Run All before commit

## Tools

- VS Code notebook diff (Source Control panel)
- `nbstripout` removes outputs automatically on commit
- A notebook with outputs is nice to read on GitHub

**Rule of this course: commit notebooks with outputs cleared, unless told otherwise.**

# 5 · AI assistants in VS Code

# Three ways to work with AI

## 1 · Autocomplete

```
def clean(df):
    return df.dropna() ⇥
```

Suggests the next lines. **Tab** to accept.

little autonomy ▮▯▯

## 2 · Chat

What does this error mean?

It says the column does not exist…

You ask, it answers. **You** decide what to copy.

some autonomy ▮▮▯

## 3 · Agent

📄 reads your files
✏️ edits them
▶ runs commands

Does the task for you. **Read the diff.**

most autonomy ▮▮▮

# Copilot and Claude in VS Code

## GitHub Copilot

- Free for verified students
- Autocomplete + chat + agent mode
- Our default for everyday tasks

## Claude Code

- Access will be provided later
- Stronger on bigger, multi-file tasks
- Keep it for tasks that need it (limited budget)

**Start with Copilot. Use Claude when the task needs it.**

# Context is everything

📄 the file you have open

▭ the code you select

@ files you mention

ℹ️ a README / instructions file

💬 your question

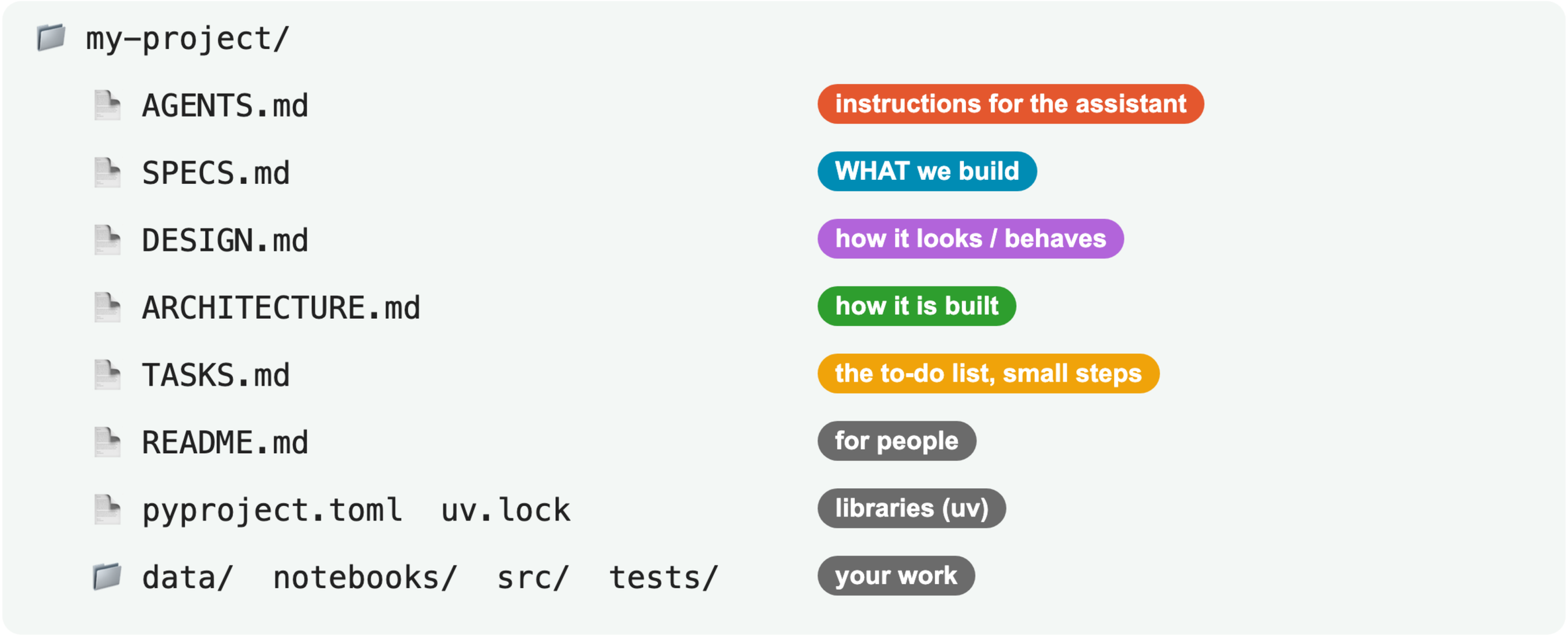
→

**The assistant**

only knows what you **show** it.

More relevant context →
better answers.

# Files of a project that uses AI



# What each file is for

## 1. AGENTS.md

- Rules and commands for the assistant
- Read at the start of every conversation

## 2. SPECS.md · DESIGN.md

- SPECS: **what** we build, acceptance criteria
- DESIGN: how it looks and behaves

## 3. ARCHITECTURE.md

- How it is built: folders, modules, data flow
- Helps the assistant put code in the right place

## 4. TASKS.md

- The to-do list in small steps
- "Work on task 2 only" = small diffs you can read

# AGENTS.md: a first version

Short and concrete. The @ lines include other files (Claude syntax):

```
# AGENTS.md
Clean the temperature file of three stations. Python 3.12, uv.
@SPECS.md
@ARCHITECTURE.md
@TASKS.md
Run tests: uv run pytest
Never edit a test to make it pass.
Work on ONE task of TASKS.md at a time.
```

# The same files, in a real small project

The temperature problem, organised for an assistant (example in the repository):

```
temperatures/
├── AGENTS.md  SPECS.md  DESIGN.md
├── ARCHITECTURE.md  TASKS.md
├── data/stations_raw.csv
├── src/clean.py  src/plots.py
├── tests/test_clean.py
└── notebooks/explore.ipynb
```

# Reading a diff

**analysis.py** +2 −1

```
  import pandas as pd
  df = pd.read_csv("readings.csv")
- print(df["temperature_c"].mean())
+ print(df["temp_avg"].mean())
+ print(df["temp_avg"].idxmax())
```

1 Which **files** changed? How many lines?

2 **Red** = removed
**Green** = added

3 Is it what I **asked**? Does that column exist?

# Four things to watch

## 1 · Invented names

a column that does not exist

`KeyError`

## 2 · Obsolete functions

learned from old tutorials

`AttributeError`

## 3 · Silent errors

a join duplicates rows

`no error at all`

## 4 · Tests rewritten

the test is changed, not the code

`all green, all wrong`

## Case 1 · An invented column

The assistant used a column that does not exist (`temp_avg`).

```
$ uv run analysis.py
KeyError: 'temp_avg'
```

## Case 2 · An obsolete function

`DataFrame.append` existed in old tutorials, but it was removed in pandas 2.0.

```
df = df.append(new_row, ignore_index=True)
AttributeError: 'DataFrame' object has no attribute 'append'
```

## Case 3 · A join that duplicates rows

No error at all, but the total changed from 220 to 320:

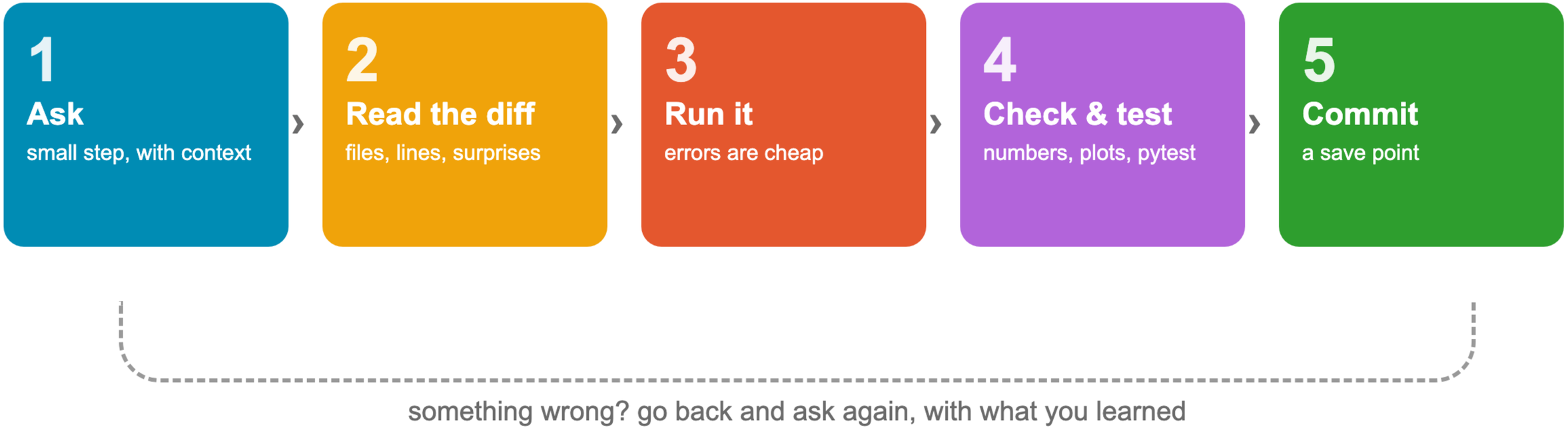
```
merged = orders.merge(customers, on="customer_id")
print("Total sales:", merged["amount"].sum())    # 320 !
# customer 1 appears twice in `customers`
```

# Case 4 · The agent changes the test

The test was red, so the 'fix' rewrote the test and broke the code:

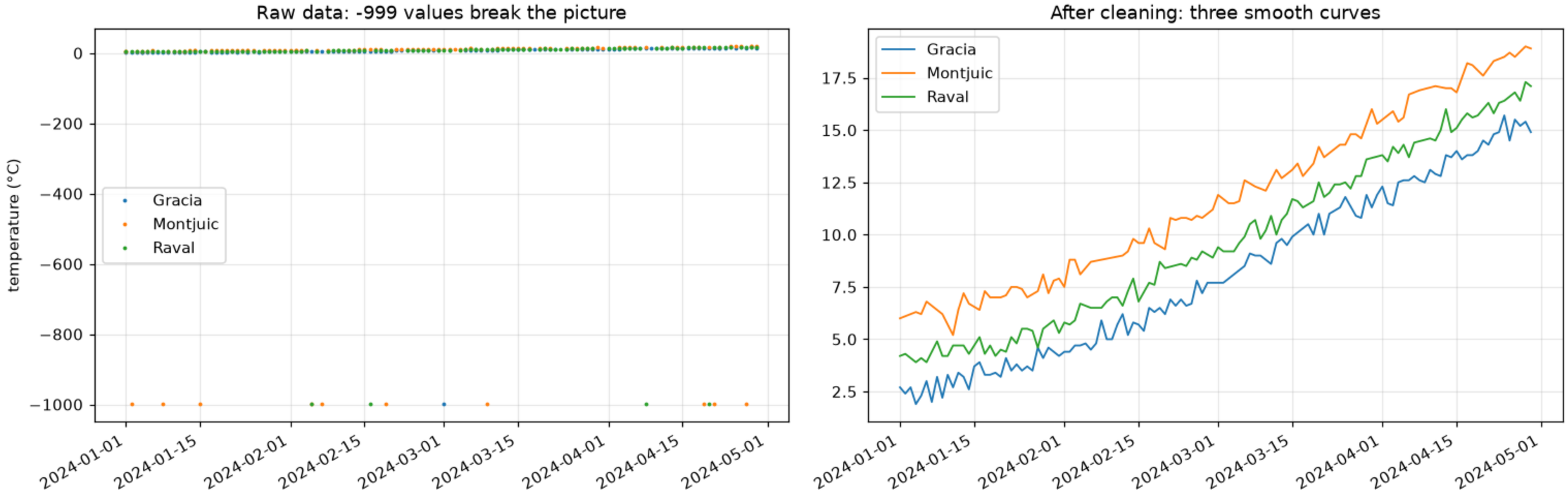
```
- assert average([2, 4, 6]) == 4
+ assert average([2, 4, 6]) == 5
- return sum(values) / len(values)
+ return sum(values) / len(values) + 1
```

# A safe loop with assistants



# 6 · Lab in pairs

# The mission: clean the temperature data




Synthetic data: 3 stations, 120 days. You will know it is clean when you see three smooth curves rising from winter to spring.

# Lab steps

## 1. Look

- Open `lab.ipynb`, list the problems in the data
- Write them in `AI_LOG.md`

## 2. Ask

- Use Copilot or Claude to write `clean_readings` in `clean.py`
- Context + small steps

## 3. Check

- **Read the diff** before accepting
- `uv run pytest`: add the missing tests

## 4. Commit

- At least **3 commits**, then push to GitHub
- Check that the commits are on github.com

# AI_LOG.md

**Keep a short log: what you asked, what failed, how you fixed it.**

# Close

# Your toolbox, in short

## 1. uv

- `uv add` · `uv sync` · `uv run`
- Commit `pyproject.toml` and `uv.lock`

## 2. Notebooks

- Kernel = `.venv`
- Restart and Run All before sharing

## 3. Git and GitHub

- Stage → Commit → **Push**
- No push, no safe

## 4. AI assistants

- Give context, ask small
- Read the diff, run it, test it

## To hand in and what comes next

- **Deliverable:** your `session02-lab` repository on GitHub
- Notebook + `clean.py` + green tests + at least 3 commits + `AI_LOG.md`
- **Next session:** ETL and EDA
- Keep your environment: it is the one we will use all along the course